Recursion — A Self-Study Guide

Work through this at your own pace. Read each section, type out the code yourself, and predict outputs before you run them. Code is in Python 3, but the ideas apply to C, C++, Java, and other languages. Every example here has been run, so the outputs shown are exactly what you'll get.

Contents: 1. What is recursion · 2. The two parts · 3. The call stack · 4. Worked examples · 5. How to write a recursion · 6. Common mistakes · 7. Self-check · 8. Assignment

---

1. What is recursion?

Recursion is when a function solves a problem by calling itself on a smaller version of the same problem — until the problem becomes small enough to answer directly.

Three words matter: **smaller** (every call shrinks the problem), **same** (it's the same kind of problem), and **obvious** (there is a smallest case you can answer without calling again).

Analogy 1 — nesting steel boxes. Picture a set of steel tiffin boxes that fit one inside another. To reach the smallest one, you open a box to find a smaller box inside, open *that* to find a smaller one, and keep going until you reach the tiniest box that holds nothing. "Open a box" is the repeated step; "the tiniest box" is where you stop.

Analogy 2 — the ticket-counter queue. You're in a long queue at a railway ticket counter and want to know your position, but you can only see the person right in front of you. So you ask them, "What number are you?" They don't know either, so they ask the person ahead of *them*. The question travels **forward** until it reaches the very first person, who can see they are **number 1**. That answer then travels **back** down the line: "You're 2"… "You're 3"… until it reaches you.

Hold on to this picture: in recursion, the question travels **forward** (calls going deeper) and the answer travels **back** (results returning). Both directions matter.

Recursion is worth using when a problem naturally contains smaller copies of itself — think family trees, folders inside folders, or splitting a task in half. For plain "repeat something 5 times," an ordinary loop is simpler.

---

2. The two parts of every recursion

Every correct recursive function has exactly two parts:

1. **Base case** — the simplest input, answered directly, with **no** further call. This *stops* the recursion.
2. **Recursive case** — shrink the input, call the function again, and **combine** that result with the current step.

Here is a function that adds up the first `n` natural numbers (for example, `1 + 2 + 3 + ... + n`):

python

```python
def total(n):
    if n == 0:          # base case: nothing left to add
        return 0
    return n + total(n - 1)   # recursive case: n + (total of the rest)
```

python

```python
print(total(5))   # 15
```

The progress rule. Every call must move *closer* to the base case. Here `n` goes `5 → 4 → 3 → 2 → 1 → 0`. If the input never shrank, the recursion would never stop.

If the base case is never reached, the program keeps calling itself until it runs out of memory for calls and Python raises a `RecursionError` (this is called a *stack overflow*):

python

```python
# WRONG: n never changes, so n == 0 is never reached → RecursionError
def total(n):
    if n == 0:
        return 0
    return n + total(n)      # bug — should be total(n - 1)
```

> [!IMPORTANT] Before trusting any recursion, ask three things: (1) Is there a base case that returns **without** calling itself? (2) Does each call use a **smaller** input? (3) Will every input actually **reach** the base case?

3. The call stack

This is the idea that makes recursion finally make sense, so read it carefully.

Each time a function is called, the computer saves a small bundle — the call's **local variables** and the note of *where to continue afterward*. This bundle is a **stack frame**, and frames pile up on the **call stack**. They are removed **last-in, first-out**, exactly like a stack of steel plates (thalis): the last plate placed on top is the first one taken off.

When a function calls itself, the new call gets its **own fresh frame** with its **own copy** of the variables. The outer call **pauses** at the point where it made the call, keeps its frame, and waits for the inner call to return — just like you waiting in the ticket queue for the answer to come back.

The example to study — notice one `print` is before the call and one is after.

python

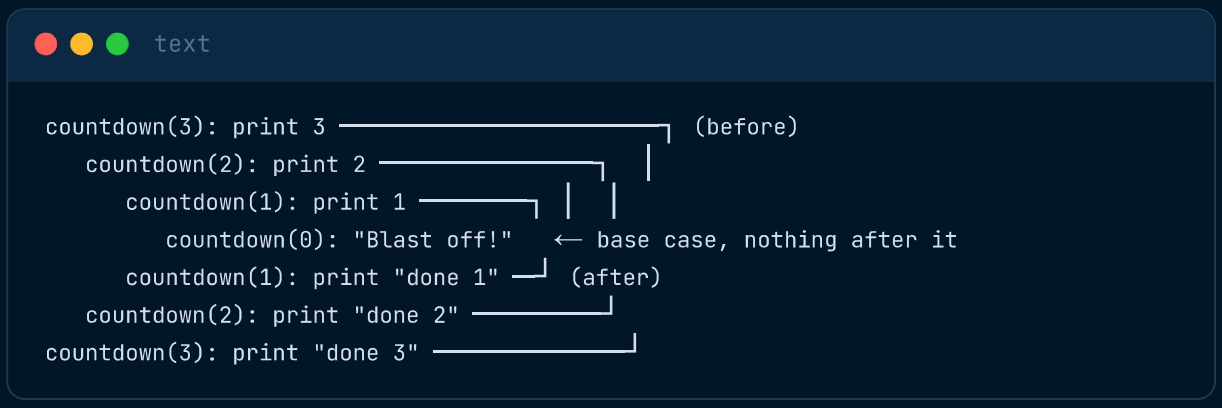
```python
def countdown(n):
    if n == 0:
        print("Blast off!")
        return
    print(n)                    # runs on the way DOWN
    countdown(n - 1)
    print("done", n)           # runs on the way UP
```

Running `countdown(3)` prints:

text

```
3
2
1
Blast off!
done 1
done 2
done 3
```

Why this order? Each call does its "before" work going down, and its "after" work coming back up:



Call	print(n) — going down	print("done", n) — coming up
countdown(3)	3	done 3
countdown(2)	2	done 2
countdown(1)	1	done 1
countdown(0)	Blast off!	— (nothing after)

> [!IMPORTANT] **Remember this:** statements **before** the recursive call run top-down (3, 2, 1). Statements **after** the recursive call run bottom-up (done 1, done 2, done 3), because the deepest paused call resumes first.

The stack is also finite — Python stops at roughly 1000 nested calls with a `RecursionError`. So recursion that goes too deep (or never ends) will crash.

---

4. Worked examples

For each, read the **insight** (how the problem relates to a smaller copy), then the **code**. Try predicting the output before checking.

4a. Factorial — build the answer on the way up

Insight. `n! = n × (n-1)!`, and `0! = 1`. (Factorials count arrangements: the number of ways 5 friends can stand in a line is `5!`.)

```python
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

```python
print(factorial(5))   # 120
```

4b. Printing in order — before vs after the call

Putting the work *before* the call prints while going down; putting it *after* prints while coming up.

```python
def print_down(n):       # prints n, n-1, ..., 1
    if n == 0:
        return
    print(n)             # before the call → descending
    print_down(n - 1)

def print_up(n):         # prints 1, 2, ..., n
    if n == 0:
        return
    print_up(n - 1)
    print(n)             # after the call → ascending
```

`print_down(4)` gives `4 3 2 1`; `print_up(4)` gives `1 2 3 4`. Same code shape, opposite order — just because of *where* the `print` sits.

4c. Sum of digits

Insight. The last digit is `n % 10`; the rest of the number is `n // 10`. So `sum_digits(n) = (n % 10) + sum_digits(n // 10)`, stopping when only one digit is left.

```python
def sum_digits(n):
    if n < 10:              # single digit: it is its own sum
        return n
    return n % 10 + sum_digits(n // 10)
```

```python
print(sum_digits(9876543210))   # 45   (digit sum of a 10-digit mobile number)
```

4d. Reverse a string

Insight. The reverse of a word is *the reverse of everything after the first letter*, then *the first letter*. A 0- or 1-letter string is already reversed.

```python
def reverse(s):
    if len(s) ⩽ 1:
        return s
    return reverse(s[1:]) + s[0]
```

```python
print(reverse("BHARAT"))   # TARAHB
```

4e. Palindrome check — you can answer early

Insight. A word reads the same both ways if its **first and last letters match** *and* the **middle** is also a palindrome. (Try **NAMAN** and **MALAYALAM**.)

```python
def is_palindrome(s):
    if len(s) ⩽ 1:
        return True
    if s[0] ≠ s[-1]:
        return False          # mismatch ⟶ answer immediately
    return is_palindrome(s[1:-1])
```

```python
print(is_palindrome("NAMAN"))       # True
print(is_palindrome("MALAYALAM"))   # True
print(is_palindrome("DELHI"))       # False
```

4f. Working through a list

Insight. Process the first element, then let recursion handle the rest. Base case: an empty list.

```python
def list_sum(prices):
    if not prices:                      # empty list ⟶ 0
        return 0
    return prices[0] + list_sum(prices[1:])
```

```python
print(list_sum([40, 25, 120, 15]))   # 200   (e.g. a ₹ grocery bill total)
```

4g. GCD (Euclid's method) — the answer passes straight back up

Insight. `gcd(a, b) = gcd(b, a % b)`, and `gcd(a, 0) = a`. Each step uses a smaller pair, so you reach `b == 0`.

```python
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

```python
print(gcd(48, 18))    # 6
```

Call sequence: `gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0) → 6`.

---

5. How to write your own recursion

Don't memorise code. Instead, answer three questions for any new problem:

> [!IMPORTANT]
>
> 1. **What is the smallest input, and what is its answer?** → that is your base case.
> 2. **If a helper already solved the problem for a slightly smaller input, how would I use that answer?** → that is your combine step.
> 3. **Does every input eventually reach the base case?** → that is termination.

The trick in question 2 is the **leap of faith**: you *trust* that the recursive call correctly solves the smaller problem, and you only reason **one level deep**. It is like delegating — you do one small piece yourself and hand the rest to an equally capable helper.

Worked example — compute a power `base^exp`.

1. Smallest input: `exp == 0` → the answer is `1` (anything to the power 0).
2. If a helper gives me `base^(exp-1)`, I just multiply it by `base`.
3. `exp` drops by one each call, so it reaches `0`. ✓

```python
def power(base, exp):
    if exp == 0:
        return 1
    return base * power(base, exp - 1)
```

```python
print(power(2, 10))   # 1024
```

You never traced the whole stack to write this — you reasoned one step and trusted the rest. That is the skill to build.

---

6. Common mistakes to avoid

Mistake 1 — missing or unreachable base case.

```python
# BUG: factorial(0) never reaches base n == 1 → RecursionError
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

Fix: use `if n == 0: return 1`, which covers `0` and everything above.

Mistake 2 — using `print` where you need `return`. `print` only *shows* a value on screen; `return` *hands it back* so the caller can use it in its calculation.

```python
# BUG: the base case prints but returns nothing (None); then n * None crashes
def factorial(n):
    if n == 0:
        print(1)
        return
    return n * factorial(n - 1)
```

Fix: `return 1` at the base case.

Mistake 3 — forgetting to combine the result.

```python
# BUG: returns the smaller answer unchanged → always 0, silently wrong
def total(n):
    if n == 0:
        return 0
    return total(n - 1)        # forgot the "n +"
```

Fix: `return n + total(n - 1)`. This one doesn't crash — it just gives wrong answers, which is harder to spot. Always hand-check a tiny input like `n = 3`.

---

7. Self-check

Try each before opening the answer. Predict the output or write the code on paper first.

Self-check A. What does this print?

```python
def f(n):
    if n == 0:
        return
    print(n * 2)
    f(n - 1)
f(3)
```

▶ **Answer**

Self-check B. What value does `mystery(4)` return?

```python
def mystery(n):
    if n == 0:
        return 0
    return 1 + mystery(n - 1)
```

▶ **Answer**

Self-check C. Write a recursive function `count_down(n)` that prints `n, n-1, ..., 1`. Then decide: should the `print` go before or after the recursive call?

▶ **Answer**

---

8. Assignment

Solve all 10. For "write a function" questions, make sure your base case and recursive case are both clear. For "output" questions, work out the order by hand before running anything. Show your working where asked.

Easy

Q1. What is the exact output of this program?

```python
def greet(n):
    if n == 0:
        return
    print("Namaste")
    greet(n - 1)

greet(3)
```

Q2. Write a recursive function `count_up(n)` that prints the numbers from `1` to `n`, each on its own line. Example: `count_up(4)` should print `1`, `2`, `3`, `4` (in that order).

Q3. In the function below, write down which line is the **base case** and which line is the **recursive case**, and explain in one sentence what the base case is for.

```python
def fact(n):
    if n == 0:
        return 1
    return n * fact(n - 1)
```

Q4. What value does this program print?

```python
def total(n):
    if n == 0:
        return 0
    return n + total(n - 1)

print(total(5))
```

Medium

Q5. What is the exact output of this program? (Watch *where* the `print` sits.)

```python
def show(n):
    if n == 0:
        return
    show(n - 1)
    print(n)

show(4)
```

Q6. Write a recursive function `digit_sum(n)` that returns the sum of the digits of a non-negative integer. Example: `digit_sum(2025)` should return `9` (because `2 + 0 + 2 + 5 = 9`).

Q7. The function below is meant to return the sum of all numbers in a list, but it always returns `0`. Find the bug, explain why it happens, and write the corrected function.

```python
def list_sum(a):
    if not a:
        return 0
    return list_sum(a[1:])
```

Q8. Using the GCD function below, what does `gcd(36, 24)` return? Show the full sequence of calls (for example, `gcd(36, 24) → gcd(24, 12) → ...`).

```python
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

Hard

Q9. Write a recursive function `is_palindrome(s)` that returns `True` if a string reads the same forwards and backwards, and `False` otherwise. Examples: `is_palindrome("NITIN")` → `True`; `is_palindrome("INDIA")` → `False`. *(State your base case clearly.)*

Q10. Write a recursive function `power(base, exp)` that computes `base` raised to the power `exp` (assume `exp` is `0` or a positive integer). Example: `power(3, 4)` should return `81`. Then answer: **how many times is `power` called in total** (including the first call) when you evaluate `power(3, 4)`? Explain your count.